Ingeniería del Software II

Unidad 1: Arquitectura y Diseño.

- **Tema 1. Arquitectura.**
  - Diseño arquitectónico.
  - Decisiones de diseño arquitectónico.
  - Organización del sistema.
  - Estilos de descomposición modular.
  - Estilos de control.
  - Arquitecturas de referencia.
  - Arquitecturas de sistemas distribuídos.
  - Arquitecturas de aplicaciones.

Tema 1. Arquitectura.

- La arquitectura del sistema afecta al rendimiento, solidez, grado de distribución y mantenibilidad de un sistema (Bosch, 2000). El estilo y estructura particulares elegidos para una aplicación puede, por lo tanto, depender de los requerimientos no funcionales del sistema:

Decisiones de diseño arquitectónico

- Basándose en su conocimiento y experiencia, los arquitectos del sistema tienen que responder a las siguientes cuestiones fundamentales:

1. ¿Existe una arquitectura de aplicación genérica que pueda actuar como una plantilla para el sistema que se están diseñando?
2. ¿Cómo se distribuirá el sistema entre varios procesadores?
3. ¿Qué estilo o estilos arquitectónicos son apropiados para el sistema?
4. ¿Cuál será la aproximación fundamental utilizada para estructurar el sistema?
5. ¿Cómo se descompondrán en módulos las unidades estructurales del sistema?
6. ¿Qué estrategia se usará para controlar el funcionamiento de las unidades del sistema?
7. ¿Cómo se evaluará el diseño arquitectónico?
8. ¿Cómo debería documentarse la arquitectura del sistema?

Decisiones de diseño arquitectónico

- ***1 Rendimiento.***

  Si el rendimiento es un requerimiento crítico, la arquitectura debería diseñarse para identificar las operaciones críticas en un pequeño número de subsistemas, con tan poca comunicación como sea posible entre estos subsistemas.

- 2 ***Protección.***

  Si la protección es un requerimiento crítico, debería usarse una arquitectura estructurada en capas, con los recursos más críticos protegidos en las capas más internas y aplicando una validación de seguridad de alto nivel en dichas capas.

- 3. Otros Aspectos a Considerar: ***Seguridad, Disponibilidad, Mantenibilidad.***

Decisiones de diseño arquitectónico

A considerar:

- ***Seguridad…..*** la arquitectura debería diseñarse para que las operaciones relacionadas con la seguridad se localizaran en un único subsistema o en un pequeño número de subsistemas.
- ***Disponibilidad….*** la arquitectura debería diseñarse para incluir componentes redundantes y para que sea posible reemplazar y actualizar componentes sin detener el sistema.
- ***Mantenibilidad…..*** la arquitectura del sistema debería diseñarse usando componentes independientes de grano fino que puedan modificarse con facilidad

Diagrama de Bloques

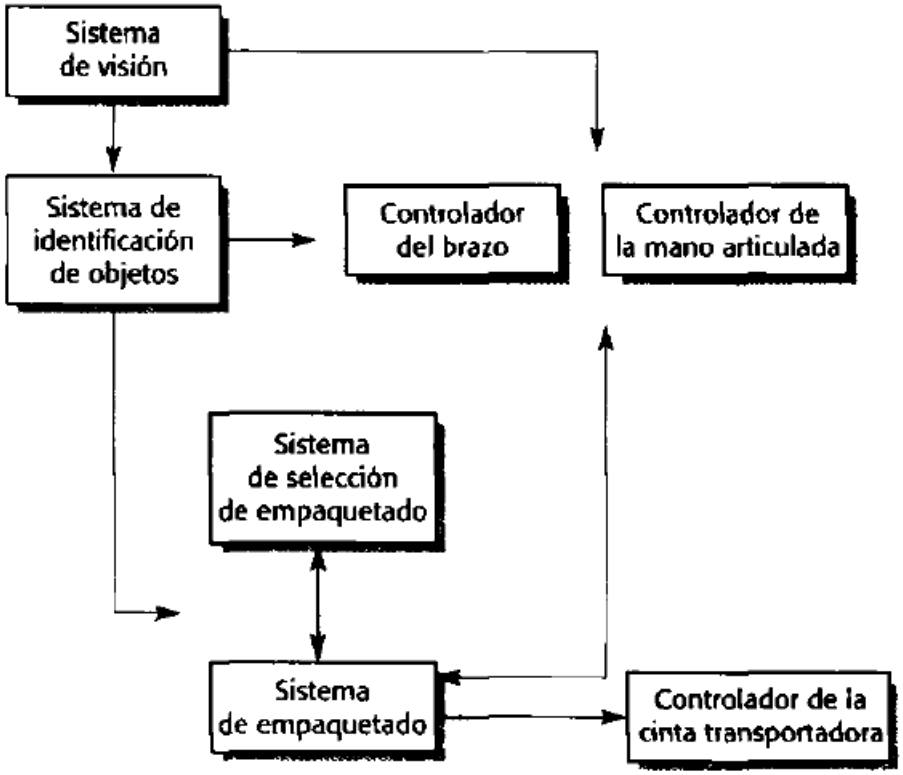


Figura 11.1 Diagrama de bloques de un sistema robótico de control de empaquetado-

Consideraciones

- Bass (Bass **et al.. 2003)** señala que los diagramas sencillos de cajas y líneas no son representaciones arquitectónicas útiles debido a que no muestran la naturaleza de las relaciones entre los componentes del sistema ni tampoco las propiedades externamente visibles de los componentes. Sin embargo, este tipo de modelo es efectivo para la comunicación con los **stakeholders** del sistema y para la planificación del proyecto debido a que no muestra los detalles.

Decisiones de diseño arquitectónico

- El resultado del proceso de diseño arquitectónico es <u>un documento de diseño arquitectónico</u>. Éste puede incluir varias representaciones gráficas del sistema junto con texto descriptivo asociado.

Decisiones de diseño arquitectónico

Los modelos arquitectónicos que pueden desarrollarse pueden incluir:

1. Un modelo estructural estático que muestre los subsistemas o componentes que han sido desarrollados como unidades separadas.

2. Un modelo de proceso dinámico que muestre cómo se organiza el sistema en procesos en tiempo de ejecución. Este modelo puede ser diferente del modelo estático.

Decisiones de diseño arquitectónico

3. Un modelo de interfaz que defina los servicios ofrecidos por cada subsistema a través de su interfaz pública.

4. Modelos de relaciones que muestren las relaciones, tales como el flujo de datos, entre los subsistemas.

5. Un modelo de distribución, que muestre cómo se distribuyen los subsistemas entre las computadoras.

Decisiones de diseño arquitectónico

Si bien cada sistema software es único, los sistemas en el mismo dominio de aplicación a menudo tienen arquitecturas similares que reflejan los conceptos fundamentales del dominio.

Decisiones de diseño arquitectónico

- Las arquitecturas de la mayoría de los sistemas grandes no utilizan un único estilo. Pueden diseñarse diferentes partes del sistema utilizando distintos estilos arquitectónicos.

- En algunos casos, la totalidad de la arquitectura del sistema puede ser una arquitectura compuesta creada mediante la combinación de diferentes estilos arquitectónicos.

Organización del sistema.

- La organización de un sistema refleja la estrategia básica usada para estructurar dicho sistema. Deben tomarse decisiones sobre la totalidad del modelo organizacional de un sistema al principio del proceso de diseño arquitectónico.

- Es frecuente que el modelo de subsistemas incluya más detalle que el organizacional.

Organización del sistema

- Los tres estilos organizacionales ampliamente usados son: un estilo de repositorio de datos, un estilo de servicios y servidores compartidos y una máquina abstracta o estilo por capas en donde el sistema se organiza en un conjunto de capas funcionales

Organización del sistema

- **El modelo de repositorio**

1. Todos los datos compartidos se almacenan en una base de datos central a la que puede acceder por todos los subsistemas. Un modelo de sistema basado en una base de datos compartida se denomina algunas veces modelo de repositorio.

2. Cada subsistema mantiene su propia base de datos. Los datos se intercambian con otros subsistemas mediante el paso de mensajes entre ellos.

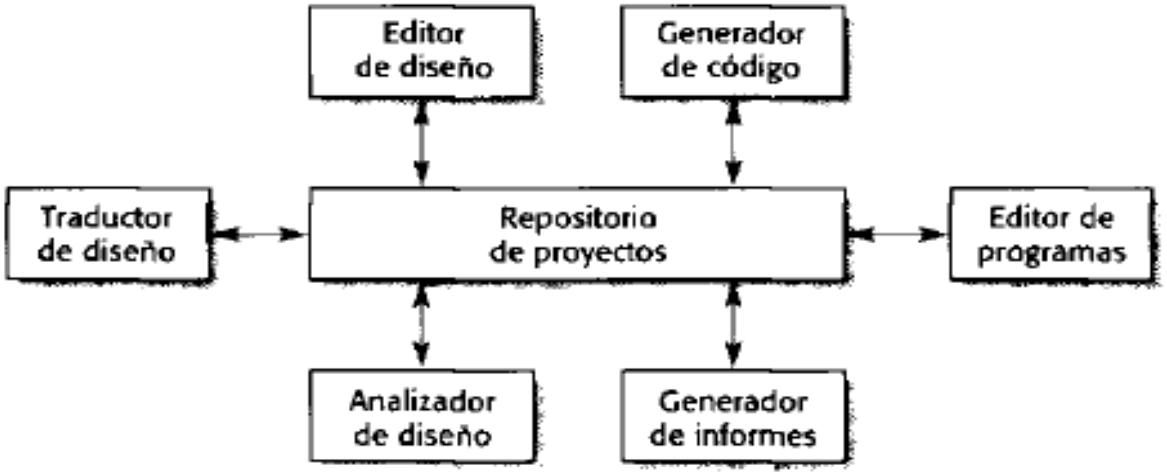
Organización del sistema

- **El modelo de repositorio**

Algunas Ventajas y desventajas:

1. Es una forma eficiente de compartir grandes cantidades de datos. No hay necesidad de transmitir datos explícitamente de un subsistema a otro.
2. Los subsistemas deben estar acordes con el modelo de datos del repositorio. Puede ser difícil o imposible integrar nuevos subsistemas si sus modelos de datos no se ajustan al esquema acordado.
3. Los subsistemas que producen datos no necesitan conocer cómo se utilizan sus datos por otros subsistemas.
4. Sin embargo, la evolución puede ser difícil traducir esto a un nuevo modelo, desde luego, tendrá un coste elevado; puede ser difícil o incluso imposible.

Organización del sistema



Organización del sistema

- **El modelo cliente-servidor.**

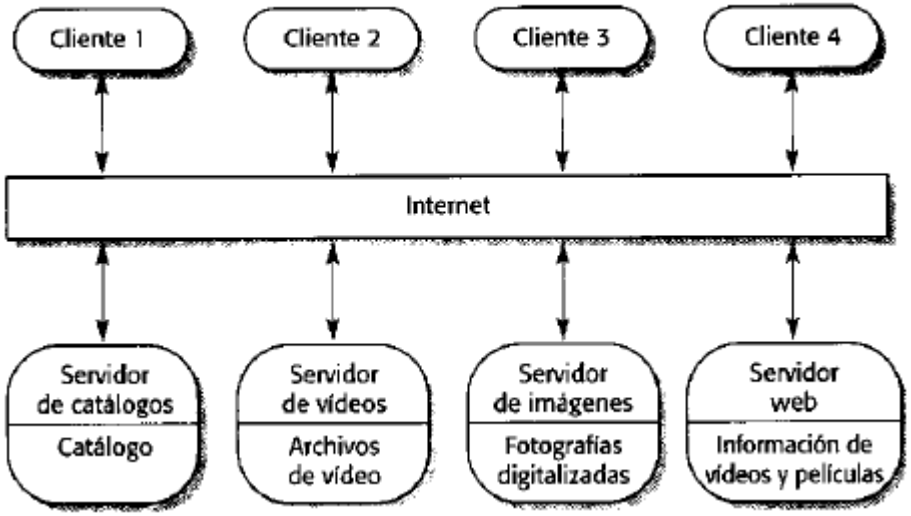
El sistema se organiza como un conjunto de servicios y servidores asociados, más unos clientes que acceden y usan los servicios.

- Los principales componentes de este modelo son:

1. Un conjunto de servidores que ofrecen servicios a otros subsistemas. Ejemplos de servidores son servidores de impresoras que ofrecen servicios de impresión.
2. Un conjunto de clientes que llaman a los servicios ofrecidos por los servidores.
3. Una red que permite a los clientes acceder a estos servicios.

Organización del sistema

- Ejemplo de un sistema basado en el modelo cliente-servidor.
- Éste es un sistema multiusuario basado en web para proporcionar una biblioteca de películas y fotografías.



Organización del sistema

- La ventaja más importante del modelo cliente-servidor es que es una arquitectura distribuida. Se puede hacer un uso efectivo de los sistemas en red con muchos procesadores distribuidos. Es fácil añadir un nuevo servidor e integrarlo con el resto del sistema o actualizar los servidores de forma transparente sin afectar al resto del sistema.

Organización del sistema

- **El modelo de capas.**

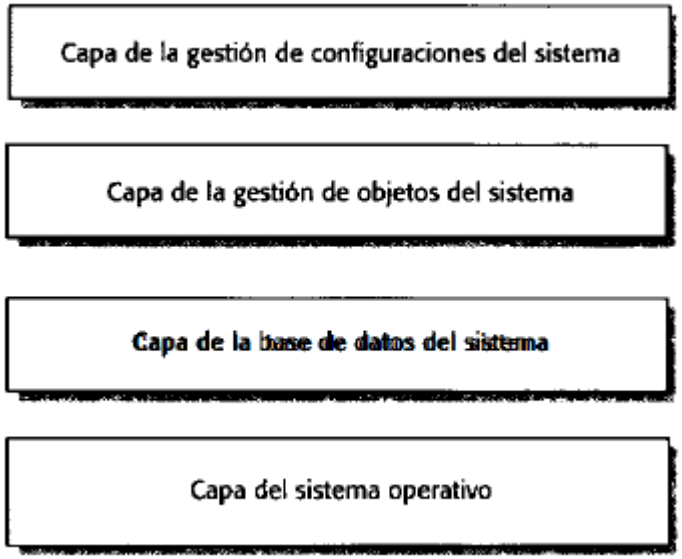
  También denominada modelo de máquina abstracta, organiza el sistema en capas, cada una de las cuales proporciona un conjunto de servicios.

  Cada capa puede pensarse como una máquina abstracta cuyo lenguaje máquina se define por los servicios proporcionados por la capa

Organización del sistema

- Un ejemplo del modelo de capas es el modelo de referencia OSI
- Otro ejemplo que ha tenido cierta influencia fue propuesto por Buxton (Buxton, 1980), quien sugirió un modelo de tres capas para un Entorno de Soporte de Programación en Ada (APSE).de protocolos de red (Zimmermann, 1980).
- El sistema de gestión de configuraciones gestiona las versiones de los objetos y proporciona facilidades de gestión de configuraciones generales.
- La desventaja de la aproximación por capas es que la estructuración de los sistemas puede resultar difícil.

Organización del sistema



Estilos de descomposición modular

- No hay una distinción clara entre subsistemas y módulos, pero resulta útil pensar sobre ellos de la siguiente forma:

1. Un subsistema es un sistema en sí mismo, cuyo funcionamiento no depende de los servicios proporcionados por otros subsistemas. Los subsistemas se componen de módulos y tienen interfaces definidas. las cuales se usan para comunicarse con otros subsistemas.
2. Un módulo es normalmente un componente de un subsistema que proporciona uno o más servicios a otros módulos. A su vez éste usa los servicios proporcionados por otros módulos. Esto no se suele considerar como un sistema independiente. Los módulos se componen normalmente de varios componentes del sistema más simples.

Estilos de descomposición modular

- Hay dos estrategias principales que se pueden usar cuando se descomponga un subsistema en módulos:

1. Descomposición orientada a objetos, en la que se descompone un sistema en un conjunto de objetos que se comunican.

2. Descomposición orientada a flujos de funciones, en la que se descompone un sistema en módulos funcionales que aceptan datos y los transforman en datos de salida.

Estilos de descomposición modular

- En la aproximación orientada a objetos, los módulos son objetos con estado privado y operaciones definidas sobre ese estado.

- En el modelo de flujos de funciones, los módulos son transformaciones funcionales.

- En ambos casos, los módulos pueden implementarse como componentes secuenciales o como procesos.

Estilos de control

- Los modelos de control a nivel arquitectónico están relacionados con el flujo de control entre subsistemas.
- Hay dos estilos de control genéricos que se usan en sistemas software:

1. **Control centralizado.** Un subsistema tiene toda la responsabilidad para controlar e iniciar y detener a otros subsistemas. También puede devolver el control a otro subsistema, pero esperará que le sea devuelta la responsabilidad del control.
2. **Control basado en eventos**. En lugar de que la información de control esté embebida en un subsistema, cada subsistema puede responder a eventos generados externamente. Estos eventos podrían provenir de otros subsistemas o del entorno del sistema.

Estilos de control

- Los estilos de control se usan conjuntamente con estilos estructurales.

- Los estilos estructurales se pueden llevar a cabo utilizando control centralizado o control basado en eventos.

Estilos de control

- Control centralizado

1. **El modelo de llamada-retorno.** Es el modelo usual de subrutina descendente en donde el control comienza al inicio de una jerarquía de subrutinas y, a través de las llamadas a subrutinas. el control pasa a niveles inferiores en el árbol de la jerarquía. El modelo de subrutinas solamente es aplicable a sistemas secuenciales.
2. **El modelo del gestor.** Este es aplicable a sistemas concurrentes. Un componente del sistema se diseña como un gestor del sistema y controla el inicio, parada y coordinación del resto de los procesos del sistema. Un proceso es un subsistema o módulo que puede ejecutarse en paralelo con otros procesos. Una variante de este modelo también puede aplicarse a sistemas secuenciales en los que la rutina de gestión llama a subsistemas particulares dependiendo de los valores de algunas variables de estado. Esto normalmente se implementa como una sentencia case

Estilos de control

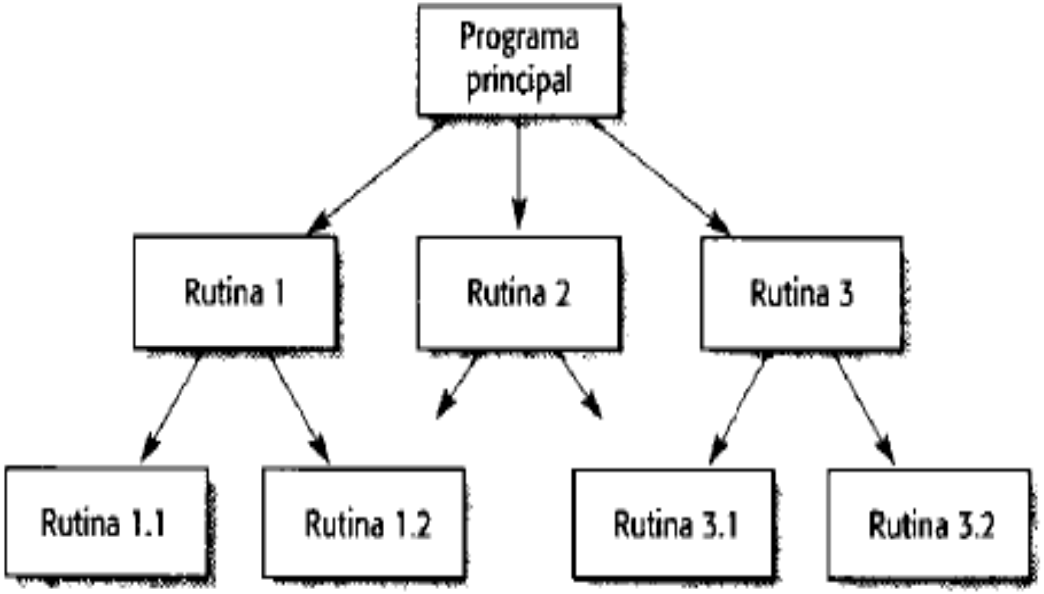


Figura 11.7 El modelo de control de llamada-retorno.

Estilos de control

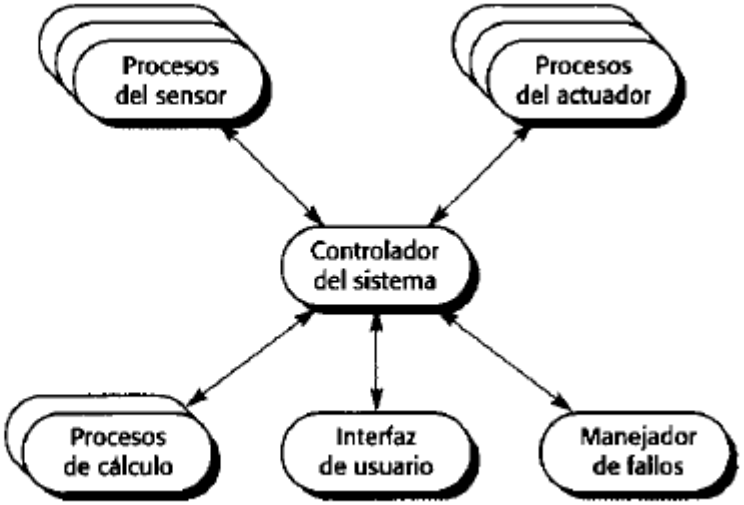


Figura 11.8
Un modelo de control centralizado para un sistema de tiempo real.

Estilos de control

- **Sistemas dirigidos por eventos**

  En los modelos de control centralizados, las decisiones de control se determinan normalmente por los valores de algunas variables de estado del sistema. Por el contrario, los modelos de control dirigidos por eventos se rigen por eventos generados externamente.

Estilos de control

- En esta sección, se analizan dos modelos de control dirigidos por eventos:

1. **Modelos de transmisión (broadcast).** En estos modelos, un evento se transmite a todos los subsistemas. Cualquier subsistema que haya sido programado para manejar ese evento puede responder a él.

2. **Modelos dirigidos por interrupciones.** Éstos se usan exclusivamente en sistemas de tiempo real, en donde las interrupciones externas son detectadas por un manejador de interrupciones. A continuación, éstas se envían a algún otro componente para su procesamiento.

Estilos de control

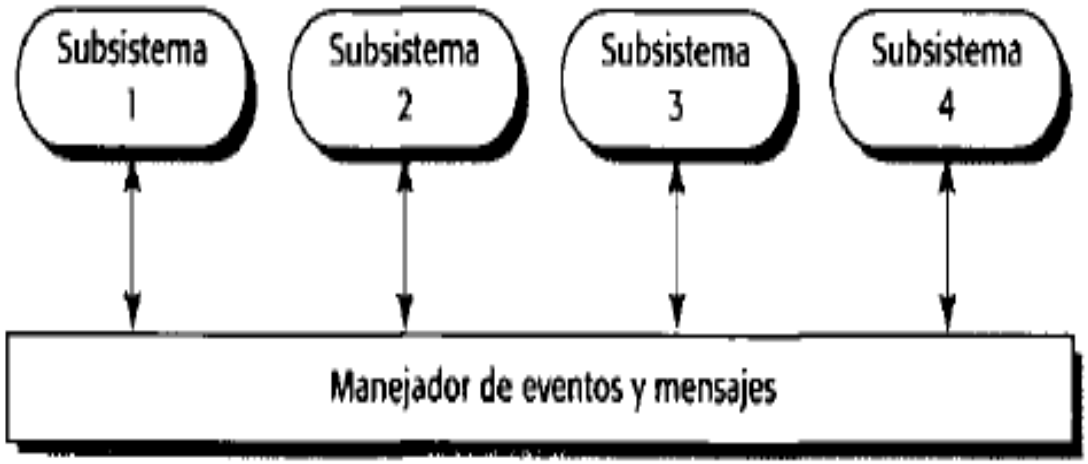


Figura 11.9
Un modelo de control basado en transmisión selectiva.

Arquitecturas de referencia

- Hay dos tipos de modelos arquitectónicos de dominio específico:

1. *Modelos genéricos*. Son abstracciones obtenidas a partir de varios sistemas reales. Encapsulan las características principales de estos sistemas. Por ejemplo, en sistemas de tiempo real, podría haber modelos arquitectónicos genéricos de diferentes tipos de sistemas tales como sistemas de recolección de datos o sistemas de monitorización.
2. *Modelos de referencia*. Son más abstractos y describen una clase más amplia de sistemas. Constituyen un modo de informar a los diseñadores sobre la estructura general de esta clase de sistemas. Los modelos de referencia normalmente se obtienen a partir de un estudio del dominio de la aplicación. Representan una arquitectura ideal que incluye todas las características que los sistemas podrían incorporar.

Arquitecturas de referencia

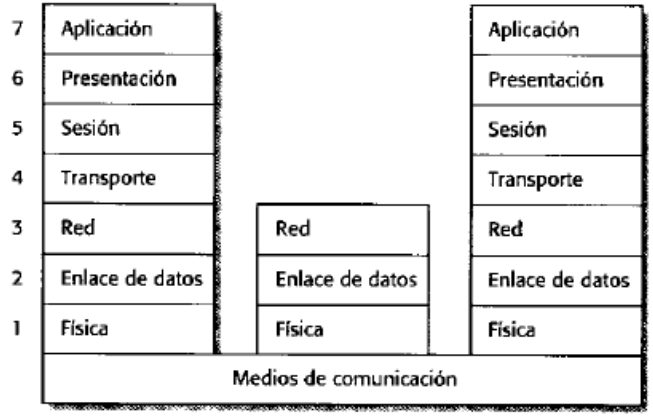


Figura 11.11
Arquitectura del modelo de referencia OSI.

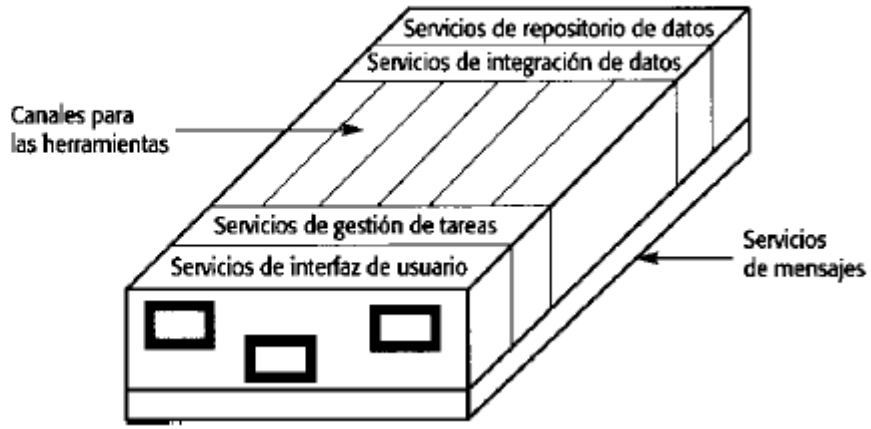


Figura 11.12
La arquitectura de referencia ECMA para entornos CASE.

Arquitectura de Aplicaciones

- **13.1 Sistemas de procesamiento de datos**
- **13.2 Sistemas de procesamiento de transacciones**
- **13.3 Sistemas de procesamiento de eventos**
- **13.4 Sistemas de procesamiento de lenguajes**

- Capítulo 13: Arquitectura de aplicaciones. - Sommerville

Arquitectura de Aplicaciones

Normalmente, los sistemas del mismo tipo tienen arquitecturas similares, y la diferencia entre estos sistemas está en la funcionalidad detallada que proporcionan. Un sistema genérico se configura y adapta para crear una aplicación específica de negocio.

Arquitectura de Aplicaciones

A continuación se describen las arquitecturas de 4 grandes tipos de aplicaciones:

1. ***Aplicaciones de procesamiento de datos.*** Las aplicaciones de procesamiento de datos son aplicaciones conducidas por los datos. Procesan datos por lotes sin intervenciones explícitas del usuario durante el procesamiento. Las acciones específicas tomadas por la aplicación dependen de los datos que se están procesando. Los sistemas de procesamiento por lotes se usan normalmente en aplicaciones de negocio en donde se realizan operaciones similares sobre grandes cantidades de datos. Dichas aplicaciones manejan un amplio rango de funciones administrativas tales como nóminas, facturación, contabilidad y publicidad.

Sistemas de procesamiento de datos

La arquitectura de los sistemas de procesamiento por lotes tiene tres componentes principales:

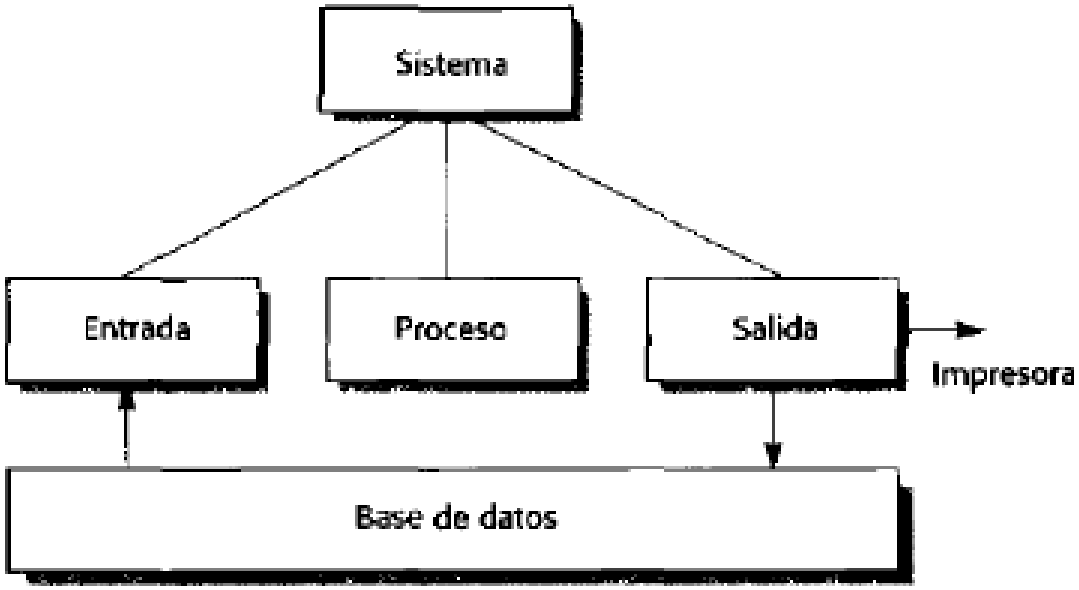
Un componente de entrada reúne entradas desde una o más fuentes.

Un componente de procesamiento realiza cálculos utilizando estas entradas; y un componente de salida genera salidas para ser escritas en la base de datos e impresas.

La naturaleza de los sistemas de procesamiento de datos, en donde los registros o transacciones se procesan en serie sin necesidad de mantener el estado entre las transacciones, implica que estos sistemas sean naturalmente orientados a funciones en vez de orientados a objetos. Las funciones son componentes que no mantienen información del estado interno de una invocación a otra.

Sistemas de procesamiento de datos

- Un modelo de entrada-proceso-salida de un sistema de procesamiento de datos.



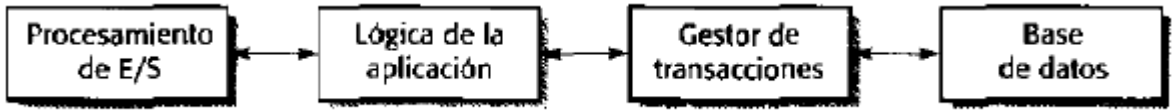
Arquitectura de Aplicaciones

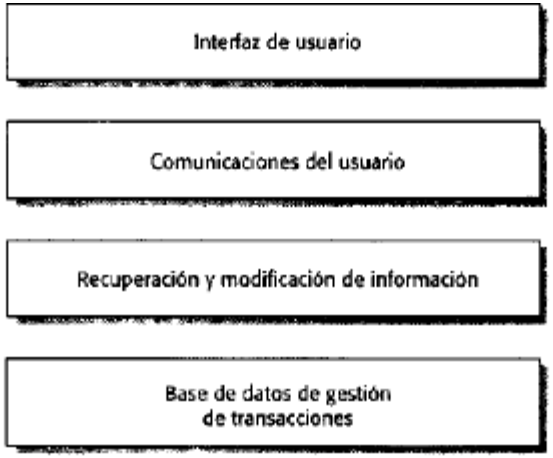
2- Aplicaciones de procesamiento de transacciones. Las aplicaciones de procesamiento de transacciones son aplicaciones centradas en bases de datos que procesan peticiones del usuario para obtener información y para actualizar la información en una base de datos. Éstos son los tipos más comunes de sistemas de negocio interactivos. Se organizan de forma que las acciones del usuario no pueden interferir entre sí y se mantenga la integridad de la base de datos.

Aplicaciones de procesamiento de transacciones.

Los sistemas de procesamiento de transacciones se diseñan para procesar peticiones de usuario a fin de obtener información de una base de datos o peticiones para actualizar la base de datos. Técnicamente, una transacción de una base de datos es una secuencia de operaciones tratada como una única unidad (una unidad atómica). Todas las operaciones de una transacción tienen que ser completadas antes de que los cambios en la base de datos sean permanentes. Desde la perspectiva de un usuario, una transacción es cualquier secuencia coherente de operaciones que satisface un objetivo. En la siguiente figura observamos la estructura de las aplicaciones de procesamiento de transacciones. Los sistemas de procesamiento de transacciones son normalmente sistemas interactivos en donde los usuarios realizan peticiones de servicios de forma asíncrona. Primero un usuario realiza una petición al sistema a través de un componente de procesamiento de E/S. La petición se procesa por alguna lógica específica de la aplicación. Una transacción se crea y se envía a un gestor de transacciones, el cual normalmente está embebido en el sistema de gestión de base de datos. Después de que el gestor de transacciones haya asegurado que la transacción se ha realizado correctamente, envía una señal a la aplicación indicando que el procesamiento ha finalizado. la estructura entrada-procesa-salida que podemos observar en las aplicaciones de procesamiento de datos también se aplican a muchos sistemas de procesamiento de transacciones.

Aplicaciones de procesamiento de transacciones.





Modelo por capas de un sistema de información

Sistemas de procesamiento de eventos

3. ***Sistemas de procesamiento de eventos.*** Las que las acciones del sistema dependen de la interpretación de eventos en el entorno del sistema. Estos eventos podrían ser la entrada de una orden por un usuario del sistema o un cambio en las variables que son monitorizadas por el sistema.

Sistemas de procesamiento de eventos

Los sistemas de procesamiento de eventos responden a eventos en el entorno del sistema o interfaz del usuario. La principal característica de los sistemas de procesamiento de eventos es que la llegada de los eventos es impredecible y el sistema debe ser capaz de tratar estos eventos cuando ocurran.

Todos nosotros usamos sistemas basados en eventos como éstos en nuestras propias computadoras -procesadores de texto, sistemas de presentaciones y juegos están todos conducidos por eventos desde la interfaz de usuario-.

Los sistemas de tiempo real, los cuales realizan acciones en «tiempo real» como respuesta a algunos estímulos externos, son también sistemas de procesamiento basados en eventos.

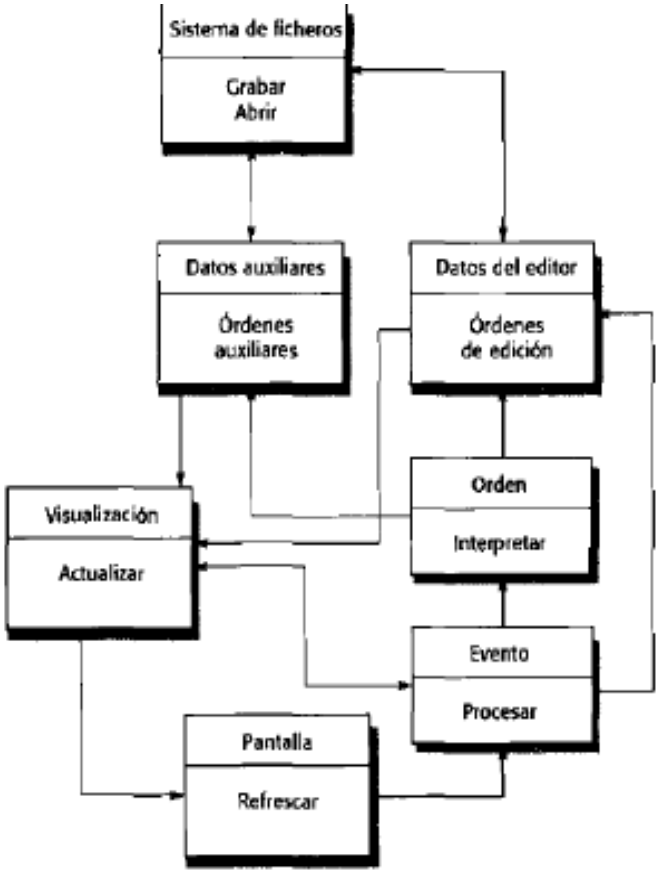
Sin embargo, para sistemas de tiempo real, los eventos no son normalmente eventos de interfaz de usuario, sino eventos asociados con servidores o actuadores en el sistema. Debido a la necesidad de respuesta en tiempo real a eventos no predecibles, estos sistemas de tiempo real se organizan normalmente como un conjunto de procesos cooperativos.

Sistemas de procesamiento de eventos

Nos centraremos en la descripción de la arquitectura genérica de los sistemas de edición. Los sistemas de edición tienen varias características que los distinguen de otros tipos sistemas y que influyen en su diseño arquitectónico:

1. Los sistemas de edición son principalmente sistemas para un único usuario. Por lo tanto, no tienen que tratar los problemas de múltiples accesos concurrentes a los datos y tienen una gestión de los datos más sencilla que los sistemas basados en transacciones.
2. Tienen que proporcionar una rápida realimentación de las acciones del usuario tales como «seleccionar» y «borrar». Esto significa que tienen que funcionar con representaciones de los datos que se almacenan en la memoria de la computadora en vez de en el disco. Debido a que los datos están en la memoria volátil, pueden perderse si hay un fallo en el sistema; por lo tanto, los sistemas de edición deberían realizar alguna previsión para la recuperación de errores.
3. Muchos sistemas de edición incluyen facilidades de recuperación que automáticamente guardan el trabajo en curso y recuperan el trabajo para el usuario en el caso de que se produzca un fallo en el sistema.

Sistemas de procesamiento de eventos



Un modelo arquitectónico de un sistema de edición. (Conjunto de objetos que interactúan)

Sistemas de procesamiento de eventos

Las responsabilidades del sistema son:

1. *Pantalla. Este objeto monitoriza el segmento de memoria de la pantalla y detecta la ocurrencia de eventos. A continuación, estos eventos se envían al objeto de procesamiento de eventos junto con sus coordenadas de pantalla.*
2. *Evento. Este objeto es disparado por un evento originado desde la Pantalla. Utiliza el conocimiento sobre lo que se está visualizando para interpretar este evento y traducirlo al comando de edición adecuado. A continuación, este comando se envía al objeto responsable de la interpretación de comandos. Para los eventos más frecuentes, tales como pulsaciones de ratón o pulsaciones de teclas, el objeto evento puede comunicarse directamente con la estructura de datos. Esto permite actualizaciones más rápidas de dicha estructura.*
3. *Orden. Este objeto procesa un comando que proviene del objeto evento y realiza una llamada al método adecuado en el objeto Editor de datos para ejecutar el comando.*

Sistemas de procesamiento de eventos

4. *Editor de datos. Cuando se llama al comando adecuado en el objeto Editor de datos, se actualiza la estructura de datos y se llama al método Actualizar enVisualización para visualizar los datos modificados.*
5. *Datos auxiliares. Además de la propia estructura de datos, los editores gestionan otros datos tales como estilos y preferencias.*
6. *Sistema de ficheros. Este objeto maneja todas las operaciones para abrir y guardar ficheros. Éstos pueden ser ficheros de datos auxiliares o ficheros del editor de datos.*
7. *Visualización. Este objeto mantiene un seguimiento de la organización de lo que se visualiza en la pantalla. Realiza una llamada al método Refrescar del objeto Pantalla cuando el contenido de la pantalla ha sido modificado.*

Arquitectura de Aplicaciones

4. ***Sistemas de procesamiento de lenguajes.*** Las intenciones del usuario se expresan en un lenguaje formal (como, por ejemplo, Java), procesan este lenguaje en algún formato interno y entonces interpretan su representación interna. Los más conocidos son los compiladores, lenguajes se utilizan para interpretar lenguajes de comandos para bases de datos, etc.

Sistemas de procesamiento de lenguajes

Los sistemas de procesamiento de lenguajes aceptan lenguaje natural o artificial como entrada y generan alguna otra representación de ese lenguaje como salida. En ingeniería del software, los sistemas de procesamiento de lenguajes más extensamente usados son los compiladores, que traducen un lenguaje de programación artificial de alto nivel a código máquina, pero otros sistemas de procesamiento de lenguajes traducen una descripción de datos en XML a comandos de consulta de una base de datos y sistemas de procesamiento de lenguaje natural que intentan traducir un lenguaje natural a otro.

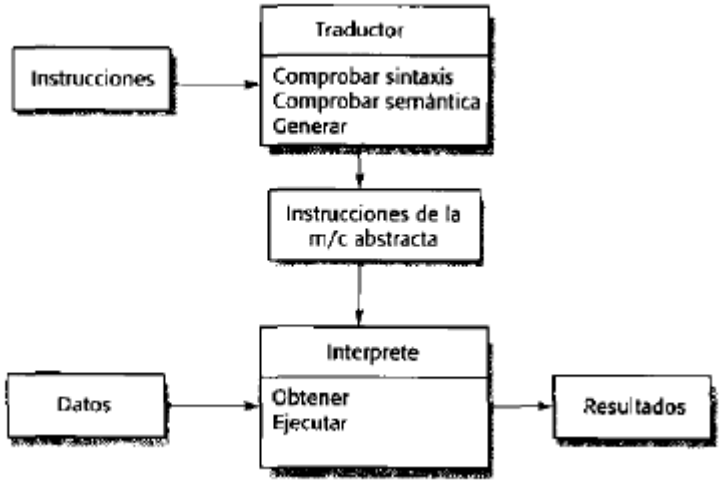
En la siguiente figura se ilustra la arquitectura de un sistema de procesamiento de lenguajes en su nivel más abstracto.

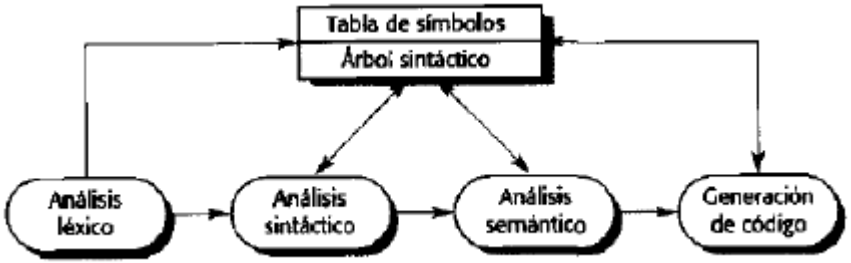
Las instrucciones describen lo que tiene que realizarse y tienen que traducirse por un traductor a algún formato interno. Las instrucciones se corresponden con las instrucciones máquina para una máquina abstracta. Estas instrucciones son interpretadas por otro componente que obtiene las instrucciones para su ejecución y las ejecuta usando, si es necesario, datos del entorno. La salida del proceso es el resultado de interpretar las instrucciones sobre los datos de entrada.

Los sistemas de procesamiento de lenguajes se usan en situaciones en las que la forma más fácil de resolver un problema es especificar esa solución como un algoritmo o como una descripción de los datos del sistema.

Los traductores en un sistema de procesamiento de lenguajes tienen una arquitectura genérica (Ver Figura).

Sistemas de procesamiento de lenguajes





Sistemas de procesamiento de lenguajes

Incluye los siguientes componentes:

1. Un analizador léxico, que toma como entrada los símbolos del lenguaje y los convierte a un formato interno.
2. Una tabla de símbolos, que almacena información sobre los nombres de las entidades usadas en el texto que se está traduciendo.
3. Un analizador sintáctico, que comprueba la sintaxis del lenguaje que se está traduciendo. Utiliza una gramática definida del lenguaje y construye un árbol sintáctico.
4. Un árbol sintáctico, que es una estructura interna que representa al programa que se está compilando.
5. Un analizador semántico, que utiliza información del árbol sintáctico y de la tabla de símbolos para comprobar la corrección semántica del texto en el lenguaje de entrada.
6. Un generador de código, que «recorre» el árbol sintáctico y genera código de máquina abstracta.

Los componentes que forman un sistema de procesamiento de lenguajes pueden organizarse de acuerdo con diferentes modelos arquitectónicos. Los compiladores pueden implementarse utilizando un modelo compuesto. Puede utilizarse una arquitectura de flujo de datos con la tabla de símbolos actuando como un repositorio para datos compartidos. Este modelo de compilación de flujo de datos todavía se usa en general. Es efectivo en entornos de procesamiento por lotes en donde los programas son compilados y ejecutados sin la interacción del usuario.

Arquitectura de Aplicaciones

Los sistemas de negocio son generalmente sistemas de procesamiento de transacciones o de datos, y la mayoría del software de computadoras personales se construye sobre una arquitectura de procesamiento de eventos. Todo el desarrollo del software se centra en los sistemas de procesamiento de lenguajes tales como compiladores. Los sistemas de procesamiento por lotes y procesamiento de transacciones se centran en bases de datos. En las aplicaciones complejas la arquitectura es la mayoría de las veces un híbrido.

PUNTOS CLAVE

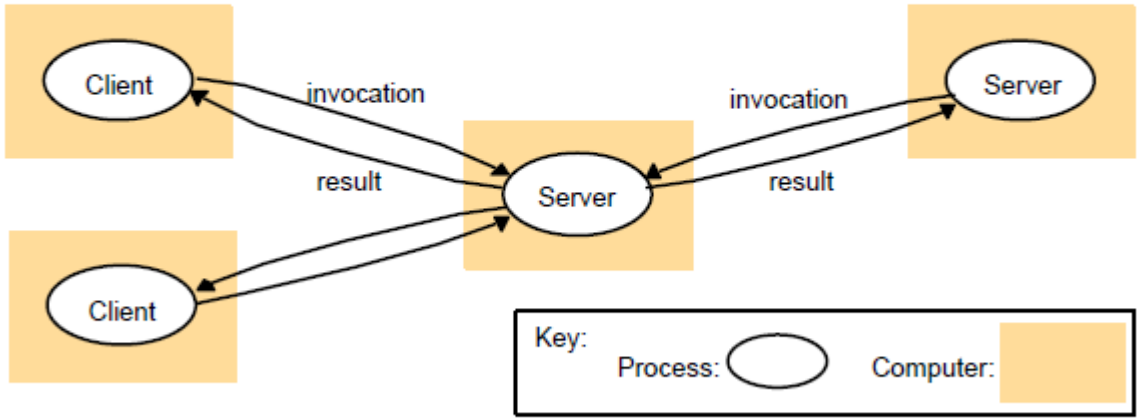
- Los modelos genéricos de las arquitecturas de sistemas de aplicaciones nos ayudan a entender el funcionamiento de las aplicaciones, comparar aplicaciones del mismo tipo, validar los diseños de los sistemas de aplicaciones y evaluar componentes a gran escala para su reutilización.
- Muchas aplicaciones pertenecen a una de las cuatro clases de aplicaciones genéricas o son combinaciones de estas aplicaciones genéricas, Los cuatro tipos de aplicaciones genéricas explicadas aquí son sistemas de procesamiento de datos, sistemas de procesamiento de transacciones, sistemas de procesamiento de eventos y sistemas de procesamiento de lenguajes.
- Los sistemas de procesamiento de datos operan en modo diferido y generalmente tienen una estructura entrada-proceso-salida. Los registros se Introducen en el sistema, la Informacl6n se procesa y se generan las salidas.
- Los sistemas de procesamiento de transacciones son sistemas Interactivos que permiten acceder y modificar la Informacl6n de una base de datos de forma remota por varios usuarios. Los sistemas de Información y los sistemas de gestl6n de recursos son ejemplos de sistemas de procesamiento de transacciones.
- Los sistemas de procesamiento de eventos Incluyen a los sistemas de edlcl6n y a los sistemas de tiempo real. En un sistema de edición, se Interpretan los eventos de la Interfaz de usuario y se modifica una estructura de datos de entrada. Los procesadores de texto y los sistemas de presentaciones son ejemplos de sistemas de edición.
- Los sistemas de procesamiento de lenguajes se utilizan para traducir textos de un lenguaje a otro y para llevar a cabo las Instrucciones especificadas en los lenguajes de entrada. Estos incluyen un traductor y una máquina abstracta que ejecuta el lenguaje generado.

Arquitectura de los Sistemas Distribuidos

- Dos modelos principales en los sistemas distribuidos

Modelo Cliente-Servidor (I)

- **Ejemplo: Servidor de ficheros**

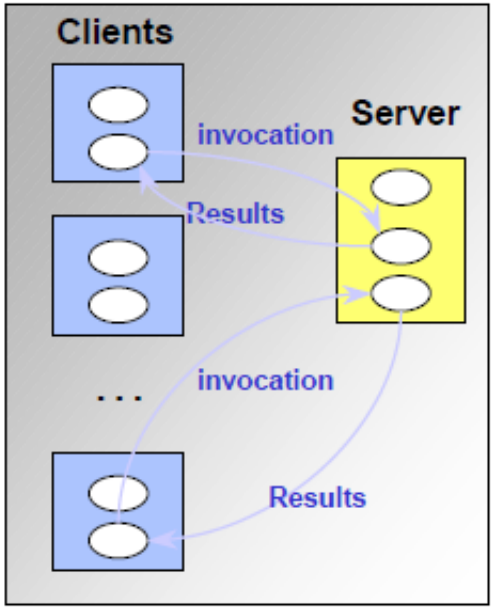


Arquitectura de los Sistemas Distribuidos

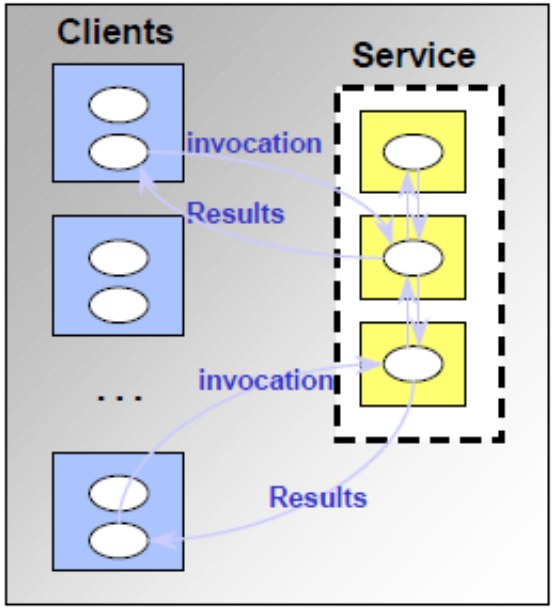
- **Cliente:**
  - Proceso que quiere acceder a datos, usar recursos o realizar operaciones en un ordenador diferente.
- **Servidor:**
  - Proceso que administra datos o cualquier otro tipo de recurso y que es capaz de hacerlos disponibles a otros procesos que están en otros ordenadores.
    - Modelo cliente-servidor también aparece en el propio ordenador
- **Interacción:**
  - Enviar petición/recibir resultado
- **Clientes ligeros/clientes pesados**
- **Ejemplos**
  - X-Window: cliente pide representación/servidor realiza representación
  - Servidor http: cliente pide página/servidor envía página

Arquitectura de los Sistemas Distribuidos

Variaciones sobre la arquitectura cliente-servidor



Un sólo servidor



Múltiples servidores

Arquitectura de los Sistemas Distribuidos

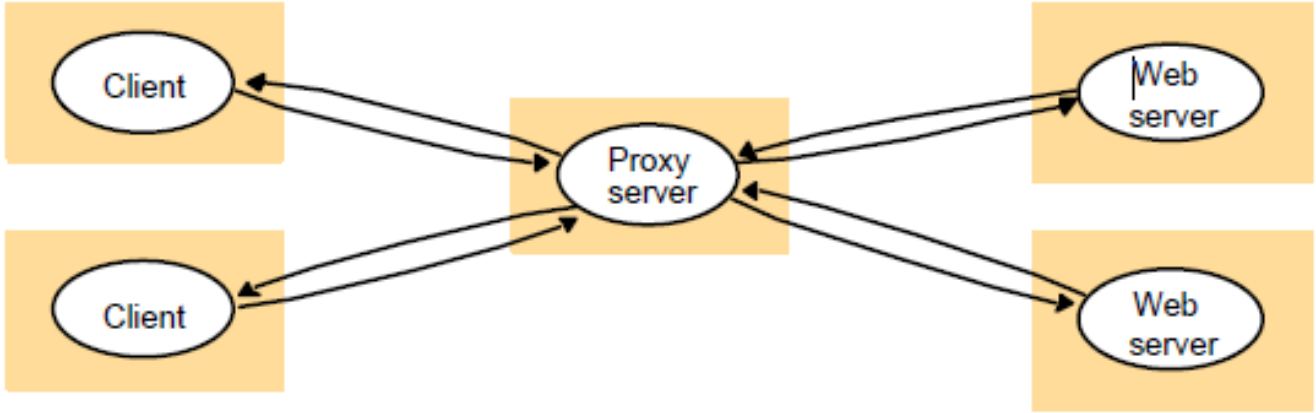
Modelo de múltiples servidores

- **Servicios son suministrados por múltiples servidores:**
  - Distribución de los recursos entre los servidores
    - Ejemplo: Web
    - Mantenimiento de los links
      - Links perdidos
  - Replicación de los recursos.
    - Servidores de alta disponibilidad: sitios de descarga , portales
- **Replicación de recursos**
  - Aumentar prestaciones
  - Tolerancia a fallos
  - Disponibilidad.
  - Problema de coordinación o de coherencia
    - La información no llega simultáneamente a todos los servidores

Arquitectura de los Sistemas Distribuidos

Modelo de servidor Proxy

- **Servidor Proxy**
  - Caché de recursos recientemente accedidos.
  - Varios niveles
    - Cliente: navegador con las páginas más recientes
    - Servidor proxy: a shared cache of resources
    - Servidor



Arquitectura de los Sistemas Distribuidos

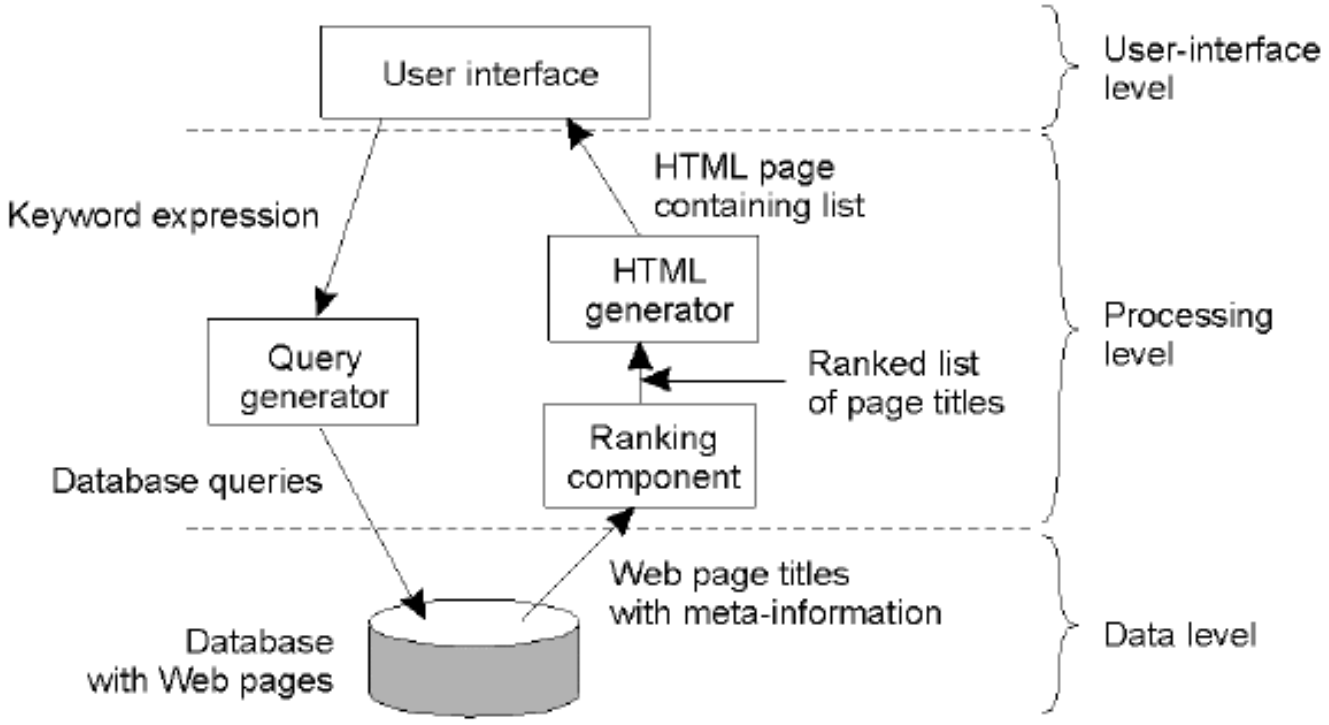
Evolución Cliente-Servidor: Modelo n-niveles

- **Cliente-Servidor clásico: 2 niveles**

Arquitectura de los Sistemas Distribuidos

Ejemplo de modelo de 3 niveles

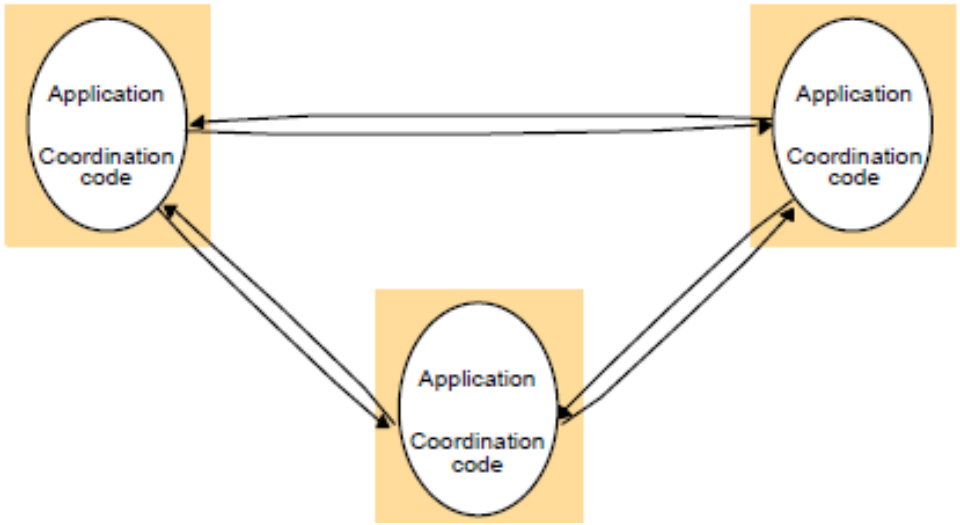
- **Búscador en Internet**



Arquitectura de los Sistemas Distribuidos

Modelo Igual a Igual (Peer-to-Peer-P2P)

- Los procesos tienen un rol similar, aunque pueden asumir un rol cliente/servidor en ciertos momentos
- Mejora la tolerancia a fallos y la escalabilidad
- Difícil de coordinar
- Ejemplos: búsqueda distribuida, enrutamiento, cálculo distribuido.



Arquitectura de los Sistemas Distribuidos

Otros modelos para estudiar los sist. distribuidos

- **Modelo de interacción**
  - Asíncrono
  - Síncrono
- **Modelo Asíncrono. No existe limitación para**
  - Velocidad de procesamiento
  - Retardo en la transmisión de los mensajes
  - Diferencia en los relojes de cada componente del sistema
  - Ejemplo: Internet
- **Modelo Síncrono**
  - El tiempo de ejecución de cada etapa está limitado: min/máx
  - Cada mensaje transmitido sobre un canal se recibe en un tiempo limitado conocido
  - Los relojes de los componentes tienen una tasa de deriva conocida
  - Ejemplo: Telemando, SCADA (Supervisory Control And Data Adquisition)

Bibliografía

- **Bibliografía Básica:**
- ‘Ingeniería de software un enfoque práctico’, Roger S. Pressman, 6th ed. Editorial MC Graw Hill. 2005 Capítulos 9- 10- 11
- ‘Ingeniería del Software’, Ian Sommerville. 7th ed. Editorial Prentice Hall. Capitulo 11 y 13.

Muchas Gracias

- Consultas.
- La próxima clase…..